

Last Resort Hotels

Analysis Report

Database Design and Implementation - Spring 2026

Ananya, Christine, Zaira, Emily (Group 6)

1. Key Design Decisions

- **Combined Room Table (No Subtypes).**
 - In Milestone 1, we modeled ROOM as a supertype with separate SLEEPING_ROOM and MEETING_ROOM subtype tables using overlapping 1:0..1 relationships.
 - For the final implementation, we merged these into a single ROOM table with nullable columns for each subtype's attributes (maxSleepingGuests, allowsRollaway for sleeping; seatingCapacity, isOutdoor, hasMoveableWalls for meeting).
 - The roomType column (SLEEPING, MEETING, SUITE, MULTI) indicates the room's function.
 - We made this change because the description explicitly states rooms can serve multiple functions: a suite can be used as a meeting room, and a meeting room with toilet facilities can be used for sleeping with rollaway beds. A single table is simpler to query, avoids unnecessary JOINS, and is more practical to implement in SQLite.
- **Customer Roles Defined by Relationships, Not Flags.**
 - Our Milestone 1 design included boolean columns on CUSTOMER: isGuest, isHost, and isBilledParty.
 - After the feedback, where this was flagged this as potentially problematic because it creates the possibility of inconsistent data: for example, a customer marked as a billed party who is never actually linked to any charges. In our revised design, we removed all role flags.
 - A customer's role is determined solely by which tables they appear in: appearing in ROOM_ASSIGNMENT means they are a guest, appearing as hostCustomerId on EVENT means they are a host, and appearing as billedPartyId on CHARGE means they are a billed party. This ensures data consistency by construction.
- **Separation of Reservation and Room Assignment.**
 - The description states that specific room assignments may not be made until the check-in date approaches, and last-minute changes are common due to breakdowns and guest plan changes.

- We modeled this as two separate entities: RESERVATION captures the customer's intent (desired dates, number of guests, linked event), while ROOM_ASSIGNMENT captures the actual allocation (specific room, actual check-in/out, extension surcharges).
 - A single reservation can produce multiple room assignments if a guest switches rooms during their stay. This design was specifically praised in our Milestone 1 feedback.
- **Beds as a Separate Table.** Our Milestone 1 design stored beds as columns (bed1_type, bed2_type) on the room table.
 - The initial feedback noted this limits flexibility and makes querying harder, finding all rooms with a king bed requires checking two columns with an OR.
 - We created a separate ROOM_BED table with roomId, bedType, and bedSize.
 - This allows a simple JOIN and WHERE filter, and scales if a room ever needs more than two beds. The constraint of maximum two beds per sleeping room is enforced at the application level.
- **Reservation Preferences as Key-Value Pairs.**
 - Our Milestone 1 RESERVATION table included preference columns: preferredBedType, preferredSmoking, preferredLocation.
 - The initial feedback noted the table was mixing structured fields with preference-like fields.
 - We created RESERVATION_PREFERENCE storing each preference as a type-value pair (preferenceType: BED_TYPE, SMOKING, POOL_PROXIMITY; preferenceValue: KING, FALSE, TRUE).
 - This allows any number of preferences per reservation and supports new preference types without schema changes.
- **Full Location Hierarchy.** We preserved the complete chain: HOTEL_COMPLEX → BUILDING → WING → FLOOR → ROOM.
 - The description specifies room numbers are only unique within wings, wings are uniquely designated within buildings, and buildings are named within complexes. Wing-level attributes (poolProximity, parkingProximity, handicappedAccess, wingSequenceNumber) are critical to reservation matching and room assignment.
 - Floor exists as a separate entity because entire floors can be designated nonsmoking. A UNIQUE constraint on (wingId, roomNumber) enforces the natural key.
- **Room Adjacency as Self-Referencing M:N.** The description defines two rooms as adjacent if there is a door between them.

- Since any room can be adjacent to multiple others and the relationship is symmetric, we modeled this as a self-referencing M:N through ROOM_ADJACENCY.
 - Both roomId1 and roomId2 are FKs to ROOM, with CHECK(roomId1 < roomId2) preventing duplicate pairs like (5,12) and (12,5).
- **Simplified Charge Table.** Our Milestone 1 CHARGE had three nullable FKs: roomId, eventId, facilityId.
 - Milestone 1 feedback noted this creates ambiguity about what a charge is associated with. We simplified to one primary FK (roomId, nullable) plus billedPartyId (NOT NULL). Event and facility context is captured through chargeType and chargeDescription rather than separate foreign keys.
- **Circular FK Handling.** CUSTOMER references ORGANIZATION (employer) and ORGANIZATION references CUSTOMER (contact person).
 - This creates a circular dependency.
 - We resolved it by creating ORGANIZATION first without a FK constraint on contactPersonId, inserting customers with organizationId = NULL, inserting organizations with valid contactPersonId references, then UPDATEing customers to set their organizationId.

2. Assumptions

1. Room subtypes are overlapping: a room can function as both sleeping and meeting. The roomType column and nullable subtype-specific columns handle this within a single table.
2. Customer roles are defined entirely by relationships. No boolean flags exist on the customer table.
3. Beds are stored in a separate ROOM_BED table. Maximum 2 beds per sleeping room enforced at application level.
4. Room numbers are unique within a wing. UNIQUE(wingId, roomNumber) enforced at the database level. Surrogate roomId used as PK for simpler FK references.
5. Reservations and room assignments are separate entities. Reservations capture preferences; assignments capture actual allocation and can change.
6. Reservation preferences stored as key-value pairs in RESERVATION_PREFERENCE for flexibility.
7. Organization always requires a contact person. Even when an org is the billed party, a named individual must be associated.
8. Room assignment priority: lowest wingSequenceNumber and lowest roomNumber meeting stated requirements.
9. Meeting room charge rules (eating slots at full rate, non-eating at half rate, one free non-eating per paid eating) are business rules enforced at the application level.
10. Sleeping room checkout: 4 PM to 12 noon. Extensions up to 2 hours free; longer extensions incur flat surcharge tracked on ROOM_ASSIGNMENT.
11. Advance deposits required based on qualificationRating. Tracked on RESERVATION.
12. Card tracking is optional. Guests choose whether to swipe. Location data may be incomplete.
13. All tables normalized to 3NF. No transitive dependencies.
14. Database represents one hotel complex (Last Resort Miami) with seed data covering operational scenarios including room switches, billing splits, events, and multiple charge types.
15. Time modeling: sleeping rooms use continuous datetime ranges (actualCheckIn/actualCheckOut). Meeting rooms use discrete TIME_SLOT references. These are intentionally different scheduling models for different room types.

3. Tradeoffs and Limitations

Nullable columns on ROOM. Merging sleeping and meeting room attributes into one table means several columns are NULL for rooms that don't use them (seatingCapacity is NULL for sleeping-only rooms, maxSleepingGuests is NULL for meeting-only rooms). We accepted more NULLs in exchange for simpler queries and fewer JOINS. The application layer validates that appropriate columns are populated based on roomType.

No subtype enforcement at the database level. The database cannot enforce constraints like "a room with hasPermanentBeds = 1 must not have seatingCapacity populated." Cross-column validation is handled at the application level. Separate subtype tables would enforce this structurally but at the cost of more complex queries.

Circular FK between Customer and Organization. Requires a multi-step insertion process (insert customers first with NULL organizationId, then organizations, then UPDATE). This adds complexity to data loading but is standard practice for circular references in relational databases.

Seed data scale. Our dataset covers one hotel complex with 3 buildings, 3 wings, 3 rooms, 7 customers, 3 events, and 3 reservations. While sufficient to demonstrate all query patterns and relationships, a production system would have significantly more data. All 23 tables are populated with coherent, interconnected data that tells a realistic operational story.

Preference matching not automated. RESERVATION_PREFERENCE stores guest preferences but the system does not automatically match preferences to room assignments. In a production system, an algorithm would compare preferences against available rooms. In our implementation, this matching is represented through the data and queries but not automated.

Event charges handled indirectly. After simplifying CHARGE to remove the eventId FK, event-related charges (meeting room usage, catering) are linked through chargeType and chargeDescription rather than a direct relationship. This is less queryable than a direct FK but avoids the ambiguity of multiple nullable FKs that our professor flagged.

Boolean representation in SQLite. SQLite has no native BOOLEAN type. We use INTEGER with CHECK(column IN (0, 1)) constraints throughout. This works correctly but requires discipline - the application must always pass 0 or 1, never TRUE/FALSE.

4. How Ambiguities in the Description Were Handled

Suite as one room or two? The description says “a suite consists of a sleeping room with an additional, separated room for meeting or working” but also says “there is only a single hallway access door to the suite.” We treated the suite as a single ROOM record with roomType = ‘SUITE’ that has both sleeping and meeting attributes populated. If there were separate hallway doors, they would be modeled as two separate rooms. This interpretation aligns with the description’s distinction: single door = one room (suite), separate doors = two rooms.

Rooms serving multiple functions. The description says “in a pinch, a meeting room could be used as a sleeping room” and mentions fold-away beds. Rather than modeling function as static, we allow the same room to be booked for either purpose depending on need. The roomType = ‘MULTI’ value covers rooms that routinely serve both functions, while the hasFoldawayBed and hasPermanentBeds flags determine what’s physically possible.

Floor existence within wings. The description says “a room number consists of one or two digits designating the floor or level, followed by two digits designating the room number on the floor.” This could mean floors are shared across wings or exist within wings. We assumed floors exist within wings (FLOOR has a wingId FK), because the room numbering scheme encodes the floor in the room number and the description discusses wing-level attributes separately from floor-level attributes (nonsmoking floors).

Advance deposit thresholds. The description lists qualifications for deposit requirements (past history, cooperativeness, flexibility, payment promptness) but does not define specific thresholds. We stored these as enum ratings (EXCELLENT, GOOD, FAIR, POOR, NEW) on CUSTOMER and left the deposit decision to business logic. The RESERVATION table tracks whether a deposit is required, the amount, and whether it’s been paid.

Billing party vs. guest distinction. The description says the billed party can be separate from the facility user, and billing can be split. We handle this with CHARGE pointing to billedPartyId (who pays) while ROOM_ASSIGNMENT tracks guestId (who stays). BILLING_SPLIT handles multi-guest splits on sleeping rooms with percentage or fixed amount splits by charge type.

What counts as a “facility”? The description mentions restaurants, health clubs, retail shops, business centers, pools, and courtyards. We created a FACILITY table to represent these, with facilityType distinguishing categories. Outdoor spaces like courtyards and pool patios are modeled as meeting rooms (with isOutdoor = 1) rather than facilities, since the description says they “are treated as meeting rooms.”

Guest location tracking scope. The description says guests “may, at their discretion, use the card to inform the computer of their current location.” We interpreted this as optional and passive, the system records scans but does not require them. CARD_SCAN_LOG captures

entries and exits, but there is no guarantee the system knows a guest's current location at any given time.

5. Evolution from Milestone 1 to Final

Our design evolved significantly between Milestone 1 and the final submission based on professor feedback and implementation experience. The key changes were:

1. Removed boolean role flags (isGuest, isHost, isBilledParty) from CUSTOMER. Roles are now defined entirely through relationships. This was the professor's primary feedback item.
2. Created ROOM_BED as a separate table, replacing bed1_type/bed2_type columns. This enables flexible querying by bed type and was a specific professor recommendation.
3. Created RESERVATION_PREFERENCE as a key-value table, replacing preference columns on RESERVATION. This addresses the professor's feedback that the reservation table was doing too much work.
4. Simplified CHARGE from three nullable FKs (roomAssignmentId, eventId, facilityId) to one primary FK (roomAssignmentId) plus billedPartyId. Event and facility context moved to chargeType and chargeDescription per professor feedback.
5. Merged SLEEPING_ROOM and MEETING_ROOM subtypes into a single ROOM table with nullable columns. This simplified implementation while preserving the overlapping subtype semantics.
6. Clarified time modeling: sleeping rooms use continuous datetime ranges, meeting rooms use discrete TIME_SLOT references. Documented this as intentional rather than inconsistent per professor feedback.
7. Added isEatingSlot column to TIME_SLOT, which was missing in Milestone 1 but needed for the eating/non-eating rate calculation business logic.
8. Resolved circular FK between CUSTOMER and ORGANIZATION by creating ORGANIZATION first without FK enforcement on contactPersonId, then using staged inserts and UPDATES.

6. Web Interface

The web interface is built with Flask (Python) and connects directly to the SQLite database. It serves as a front-desk operations and management reporting tool for Last Resort Hotels. We decided to focus on front-desk operations instead of a client-facing reservation system because we agreed that the different attributes and tables (e.g. the specifications of bedroom suites and meeting rooms) would be best displayed in a management perspective.

Scope Decision: Management-Facing Interface. Early in the project, we developed wireframes for two distinct audiences: hotel guests (a customer-facing booking and account view) and hotel staff (a management and operations view). After review, we decided to build only the management-facing interface for the final submission.

A customer-facing portal would require its own authentication layer, personalized data access controls, and a fundamentally different information architecture; all would have diluted the quality of the management side. The project showcases how a database surfaces meaningful, aggregated information. That goal is better served by a staff dashboard showing revenue trends, occupancy rates, and guest history across the whole system than by a guest-facing page that only exposes a single user's bookings.

Interface Structure. All pages share a common base layout (base.html) with a persistent header, navigation bar, and footer. Operational tasks (adding guests, making reservations, changing rooms) are accessible from the header; analytical views (the dashboard and reports) are accessible from the nav. Pages fall into three functional categories.

Operational pages handle day-to-day front desk tasks. The guest management page (guests.html) displays every guest with their stay count, current in-house status, and quick actions for viewing history or deleting a record. The room change page and add-guest/add-reservation forms support common workflows without requiring navigation away from the main interface.

Showcase pages (rooms_showcase.html, amenities_showcase.html) display live inventory pulled from the database including room types, availability status, base rates, and capacity, presented in a card grid. Counts and rates are query-driven and reflect the actual state of the database.

The dashboard and reports are the analytical core. The dashboard (dashboard.html) uses Chart.js to render four live visualizations: a revenue growth line chart over time, a doughnut chart of current occupancy breakdown, a revenue-by-charge-type doughnut, and a guest loyalty distribution bar chart. A ranked list of top guests by stay count and spend appears below the charts. All chart data is passed from the backend as JSON and rendered client-side. The reports section provides a sidebar layout with parameterized filters (date range, capacity, guest selector) depending on the query being shown.

Design Decisions and Tradeoffs. Rather than building a separate HTML page for each query, we used a single report.html template that accepts column definitions, row data, and an optional filter form type from the backend. This kept the frontend consistent and made it straightforward to add new reports without modifying existing HTML. The interface does not support live filtering or dynamic table updates outside of form submissions; adding client-side interactivity would have added complexity without meaningfully improving the demonstration of the underlying database queries.

Technology stack. Flask for routing and templating, SQLite for the database, Jinja2 for HTML rendering, and standard CSS for styling. Each route opens the database, executes one or more queries, and passes results to an HTML template.

Key features. The interface includes a search function for guests and rooms, a Reports dropdown for query-driven views, a New Customer form for registering guests/hosts/billing contacts, and a New Reservation form for booking sleeping rooms. The reservation form includes guest selection, event linking, date picking, number of guests, status, and deposit tracking.

Query integration. All 13 queries are accessible through the web interface, either as dedicated report views or integrated into operational pages. Queries use JOINS across multiple tables, WHERE clauses for filtering, GROUP BY for aggregation, and ORDER BY for relevance sorting. Results render as HTML tables.

Design philosophy. The interface prioritizes functional clarity over visual polish, consistent with the project requirement that “the focus is system modeling, data organization, and interpretation, not branding, marketing, or visual polish.” The interface demonstrates how structured data can be queried, summarized, and presented to support hotel operations.

Limitations. The customer-facing wireframe we initially designed, which included a guest portal for viewing personal reservations and charges, was not implemented. If the system were extended, that would be the natural next step, paired with session-based authentication. The interface also has no real-time update mechanism; the dashboard reflects the state of the database at page load rather than streaming live updates.

7. Queries Overview

We implemented 13 queries covering daily operations and management reporting. All queries are designed around multi-table joins and aggregation, ensuring that each result reflects live, relational data rather than isolated snapshots. The organizational structure, grouped into four functional categories, mirrors the departmental workflows the queries are meant to support, making it straightforward to extend or adapt individual queries as operational needs evolve:

Room Operations (Q1, Q2, Q8, Q9, Q10, Q11). Currently occupied rooms with guest info, rooms available right now, smoking rooms near pool sorted by rate, available rooms with amenities for a date range, rooms that can sleep a specific number of guests, and bed types available per room. These support front-desk staff in real-time room management and guest requests.

Billing and Finance (Q3, Q4). Total charges per customer sorted by highest spend, and unpaid/pending charges. These support the accounting team in tracking revenue and following up on outstanding balances.

Events and Reservations (Q5, Q6, Q7). Upcoming events with host name and estimated attendance, rooms booked per event, and reservations missing required deposits. These support event coordination and deposit collection workflows.

Guest History (Q12, Q13). Full stay history for each guest with per-stay charge totals, and a directory of all facilities at the complex. These support relationship management and guest services.